

Analizadores de código

Revisión estática y configuración

¿Qué es un analizador de código?

Un analizador de código lee tu código fuente y detecta problemas SIN ejecutar el programa. Busca errores típicos, malas prácticas y problemas de calidad mirando el texto del código — igual que un corrector ortográfico, pero para lógica de programación.

La clave es que actúa ANTES de que ejecutes nada. No espera a que el programa falle en tiempo de ejecución — te avisa en el momento en que escribes que algo huele mal.

Análisis estático vs. ejecución

	Análisis estático	Ejecución del programa
Cuándo actúa	Mientras escribes / al guardar	Cuando arrancas el programa
Qué necesita	Solo el código fuente	Compilar + datos de prueba
Qué detecta	Patrones de error conocidos	Errores que ocurren de verdad

¿Por qué se usan en proyectos reales?

Imagina un equipo de cinco personas: cada una con su estilo, sin reglas comunes. El código se convierte en un puzzle. Los analizadores definen reglas objetivas que se aplican igual para todos.

Prevención de bugs

Detectan fallos típicos antes de ejecutar.
Un NPE o un recurso sin cerrar aparece subrayado antes de que el programa arranque.

Estilo uniforme

Todo el equipo sigue las mismas reglas.
No hay debates de cómo nombrar las variables: lo decide la regla.

Mantenibilidad

Código más fácil de leer y modificar.
Un método de 200 líneas con 15 if anidados es difícil de cambiar sin romper algo.

Integración con CI/CD

En empresas, el análisis se ejecuta automáticamente al hacer un push y puede bloquear un merge si hay problemas graves.

Tipos de problemas que detectan

Los analizadores agrupan sus avisos por tipo. Entender la diferencia ayuda a priorizar qué corregir primero:

Tipo	Qué busca	Ejemplo concreto
Bug probable	Patrones que casi siempre causan un error en tiempo de ejecución	Llamar a un método sobre una variable null
Código muerto	Código que nunca se ejecuta o asignaciones redundantes	Variable que se asigna pero nunca se lee
Mala práctica	Construcciones que funcionan pero son frágiles o confusas	Comparar strings con == en vez de .equals()
Estilo	Convenciones de formato y nomenclatura	Nombre de clase en minúscula, línea demasiado larga
Complejidad	Métodos o clases demasiado complejos para mantenerse	Método de 100 líneas con 10 condiciones anidadas

Ejemplo: NullPointerException (NPE)

NPE = NullPointerException: intentar usar una variable que no apunta a ningún objeto (null). Compila sin errores, pero el programa explota en tiempo de ejecución. El analizador lo detecta ANTES.

❌ Peligroso — lanza NPE

```
String rol = null;

// Llama a equals() sobre null
// → NullPointerException
if (rol.equals("ADMIN")) {
    System.out.println("Es admin");
}
```

✅ Seguro — el literal no puede ser null

```
String rol = null;

// El literal nunca es null
// → seguro aunque rol sea null
if ("ADMIN".equals(rol)) {
    System.out.println("Es admin");
}
```

El analizador detecta que rol puede ser null en el momento de la llamada y subraya la línea antes de que ejecutes nada.

Ejemplo: número mágico

Un número mágico es un valor literal sin nombre que aparece en el código. ¿Qué significa ese 8? ¿Mínimo de caracteres? ¿Máximo? ¿Un límite de intentos? El analizador avisa para que lo conviertas en una constante con nombre.

✗ Número mágico — intención opaca

```
// ¿Por qué 8?  
// ¿mínimo? ¿máximo? ¿intentos?  
if (password.length() < 8) {  
    System.out.println(  
        "Contraseña corta");  
}
```

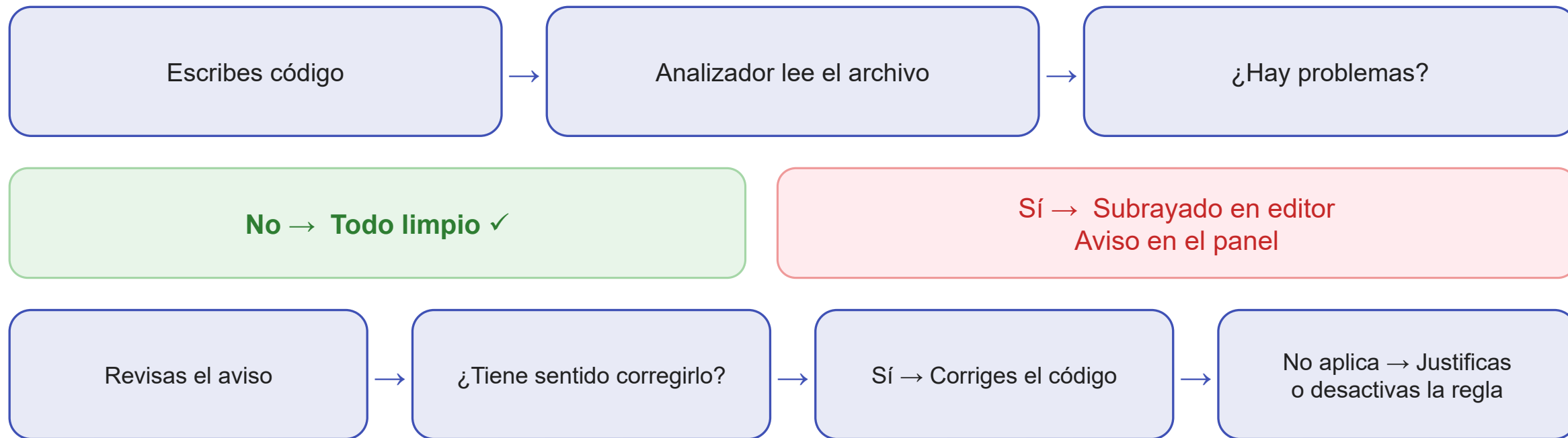
✓ Constante — intención clara

```
final int MIN_PASSWORD_LENGTH = 8;  
  
// Intención explícita  
if (password.length()  
    < MIN_PASSWORD_LENGTH) {  
    System.out.println(  
        "Contraseña corta");  
}
```

Si el mínimo cambia a 10 en el futuro, solo cambias la constante en un sitio — no buscas todos los 8 dispersos por el código.

Ciclo de análisis — el analizador siempre está activo

El analizador no es algo que lanzas una vez al terminar — revisa el archivo cada vez que guardas o escribes:



Un aviso no es una orden — es una pregunta. A veces el analizador se equivoca o la regla no aplica a tu caso. Lo importante es que la decisión sea consciente, no que ignores todos los avisos porque molestan.

Herramientas habituales en Java

No son excluyentes — en proyectos de empresa se usan varias a la vez, cada una con su función:

Herramienta	Punto fuerte	Cuándo usarla
IntelliJ Inspections	NPE probables, código muerto, simplificaciones automáticas	Siempre: ya viene integrada, sin instalar nada
SonarQube for IDE	Bugs, seguridad y malas prácticas con explicación detallada	En clase y proyectos: plugin gratuito, muy completo
Checkstyle	Convenciones de estilo: nombres, longitud de línea, imports	Cuando el equipo quiere estilo uniforme en fichero de reglas compartido
PMD	Malas prácticas de diseño: código duplicado, complejidad, variables sin usar	Para analizar calidad de diseño más allá del estilo
SpotBugs	Bugs por patrones: comparaciones peligrosas, recursos sin cerrar	Cuando se quiere detectar errores reales, no solo estilo

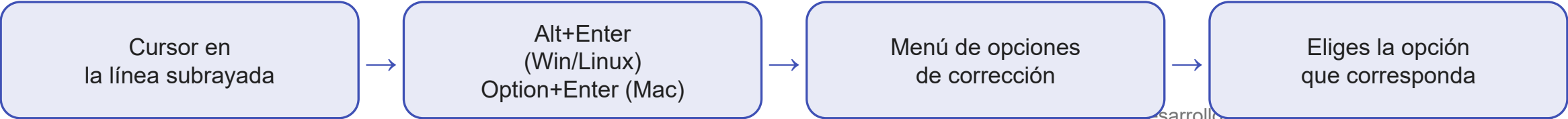
En clase: con IntelliJ Inspections y SonarQube for IDE tienes más que suficiente. Las otras tres las verás en prácticas en empresa integradas en el pipeline de CI/CD.

Cómo se ven los avisos en IntelliJ

IntelliJ marca los problemas de varias formas sin que tengas que hacer nada:

Zona del IDE	Qué muestra
Editor — subrayado amarillo	Warning: posible problema, no siempre un error
Editor — subrayado rojo	Error probable o error de compilación
Barra de scroll (derecha)	Marcas de color en miniatura: dónde están los problemas sin desplazarte
Esquina superior derecha (semáforo)	Verde = limpio · Amarillo = warnings · Rojo = errores probables. Clic para navegar.
Analyze → Inspect Code	Informe de todo el proyecto agrupado por categoría. Elige ámbito (archivo / módulo / proyecto)

Aplicar correcciones rápidas (quick fix)



Configuración: qué se puede ajustar

Los analizadores no son todo o nada — se configuran para que encajen con el proyecto:

Aspecto	Qué permite hacer
Severidad	Cada regla tiene nivel Error / Warning / Info. Se puede cambiar la prioridad de cada regla según el criterio del equipo.
Reglas activas	Puedes activar o desactivar reglas concretas. Si una genera muchos falsos positivos, se desactiva — con criterio, no por comodidad.
Exclusiones	Las carpetas build/, target/ y código generado no deben analizarse. Se excluyen para no mezclar ruido con avisos reales.
Config compartida	La configuración va en un fichero del repositorio (checkstyle.xml, sonar-project.properties). Todos trabajan con las mismas reglas.

Antipatrón: ignorar todos los avisos porque molestan. Mejor 10 reglas bien elegidas que 200 que nadie lee.

SonarQube for IDE (antes SonarLint)

SonarQube for IDE (renombrado en 2024, antes llamado SonarLint) es el plugin de análisis estático más completo para IntelliJ. Es gratuito, open-source, y funciona directamente en el editor sin ningún servidor.

Lo que lo diferencia de las inspecciones de IntelliJ: por cada aviso muestra POR QUÉ es un problema, qué riesgo tiene y cómo corregirlo — no solo un mensaje críptico.

Dos modos de uso

Standalone Mode — en clase

Funciona sin conectarse a ningún servidor.

- Detecta bugs, malas prácticas y seguridad en más de 20 lenguajes
- Avisos en el panel inferior y el editor
- Completamente gratuito, sin cuenta

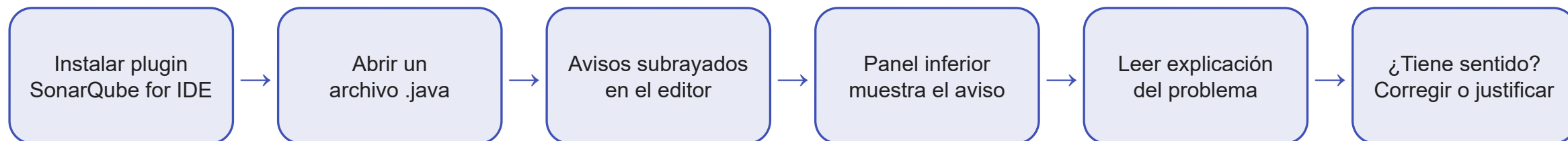
Connected Mode — en empresa

Se conecta a un servidor SonarQube del equipo.

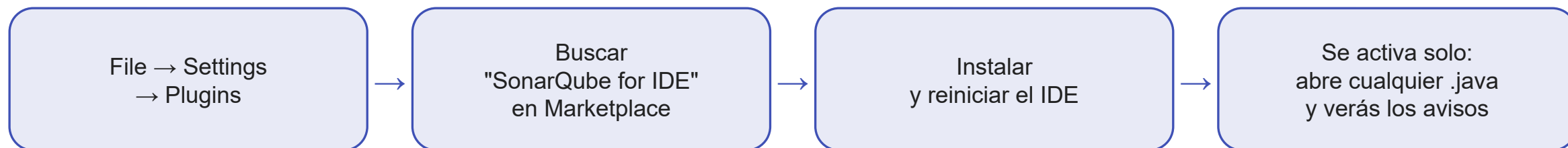
- Todos comparten las mismas reglas del servidor
- El IDE sincroniza reglas y exclusiones automáticamente
- Los avisos del IDE coinciden con el CI/CD

SonarQube for IDE — flujo e instalación

Flujo en Standalone Mode



Cómo instalar el plugin



¿Ves 'SonarLint' en tutoriales de internet? Es el mismo plugin, renombrado en 2024. Las instrucciones siguen siendo válidas.

Connected Mode: File → Settings → Tools → SonarQube for IDE → Add connection. Necesitas URL del servidor y token de acceso.

Cómo gestionar los avisos sin agobiarte

Cuando abres un proyecto y el analizador muestra 47 avisos, la reacción natural es ignorarlos todos. Hay una forma mejor:

1

Clasifica por severidad, no por orden de aparición

Empieza por errores probables (rojo), luego warnings (amarillo), deja el estilo para el final. Un NPE potencial es más urgente que una línea de 85 caracteres.

2

Distingue 'el código está mal' de 'la regla no aplica aquí'

No todos los avisos son correcciones obligatorias. Si en tu contexto concreto no es un problema real, desactiva la regla o márcalo como Won't fix con justificación.

3

Reduce el ruido desde el principio

Excluye target/, build/ y código generado. Si el analizador avisa de 30 problemas en código que no tocas, estás perdiendo la señal entre el ruido.

4

Acuerda las reglas en equipo

Pon la configuración en el repositorio. Así nadie se sorprende cuando el CI rechaza un merge que en su IDE parecía limpio.